

RV32I 5-Stage Pipelined Processor

A Complete Verilog Implementation with Harvard Architecture

Project Overview

This project implements a complete 5-stage pipeline RISC-V (RV32I) processor in Verilog. The design follows classic computer architecture principles with separate instruction and data memory (Harvard architecture), integrated hazard detection, data forwarding, and proper branch/jump handling. The processor successfully executes real C programs compiled with the RISC-V GNU toolchain, verified through simulation with complex algorithms like Bellman-Ford graph traversal.

Key Features

Feature	Details
Architecture	5-stage pipeline (IF → ID → EX → MEM → WB)
ISA Support	RV32I base instruction set (30+ instructions)
Memory Model	Harvard architecture with separate IMEM & DMEM
Hazard Handling	Load-use detection + data forwarding (2 paths)
Branch Logic	Delayed branch slots + branch prediction
Jump Support	JAL, JALR with proper return address handling
ALU Operations	Arithmetic, logical, shift, comparison (14 operations)
Simulation	Verified with real C programs (no libc dependency)

Pipeline Architecture

The processor implements a classic 5-stage pipeline where each stage processes one instruction in parallel. This significantly improves throughput compared to a single-cycle or multicycle design. However, it introduces data and control hazards that must be carefully handled to maintain correctness.

Pipeline Stages

Stage 1: Instruction Fetch (IF)

Retrieves the next instruction from instruction memory using the program counter (PC). The PC is updated based on branch/jump decisions from previous stages. Basic sequential fetch follows $PC \leftarrow PC + 4$, while branches and jumps override this.

Stage 2: Instruction Decode (ID)

Decodes the fetched instruction to extract operation type, register operands, and immediate values. Reads register file values for rs1 and rs2 (source operands). The controller generates control signals based on opcode and funct3/funct7 fields. This stage also performs sign extension on immediate values for I/S/B type instructions.

Stage 3: Execute (EX)

Performs the actual computation: ALU operations, comparisons, or address calculations. Uses data forwarding to bypass earlier pipeline stages when needed (e.g., an instruction depends on the result of the immediately preceding instruction). For branches, compares operands and signals jumps to the controller. For load/store, calculates the memory address.

Stage 4: Memory Access (MEM)

Handles all memory transactions: loads read from data memory, stores write to data memory. Passes through ALU results for non-memory instructions. Manages byte-select logic for different load/store widths (lw, lb, lbu, etc.).

Stage 5: Write Back (WB)

Updates the destination register with the final result (from ALU or memory). Does not write for store instructions or comparison operations. The write-enable signal (WE) controls whether to update the register file.

Hazard Handling & Data Forwarding

Pipelined execution creates three main types of hazards: structural (not present here, no shared resources), data (RAW—Read After Write), and control (branches/jumps). The processor implements:

Data Hazards (Load-Use)

When the current instruction depends on a load result that's still in the pipeline, we cannot forward (the data isn't ready yet). The hazard detection unit stalls the pipeline for one cycle by freezing the IF/ID and ID/EX latches and injecting a NOP (no-op) into the EX stage. After one stall, the loaded data becomes available in the

EX stage and can be forwarded.

Data Forwarding

For most data dependencies (non-load), results are forwarded directly from earlier stages. The forwarding unit checks if the destination register of an EX or MEM stage instruction matches the source operands of the current ID stage instruction. If so, the result is bypassed directly into the ALU inputs, eliminating unnecessary stalls. This allows consecutive dependent instructions to execute with minimal delay.

Control Hazards (Branches & Jumps)

Branch instructions (beq, bne, blt, etc.) make decisions about the next instruction address. In this design, branches stall the pipeline for one cycle (the branch slot). During that cycle, the branch is evaluated in the EX stage, and if taken, the PC is updated. Jump instructions (jal, jalr) similarly cause a one-cycle stall. This delayed-branch approach is simpler than dynamic branch prediction but adds a known latency.

Module Architecture

The design is composed of modular components, each with a specific role in the pipeline. Here's a breakdown of the key modules and their interactions:

IF_MODULE: Instruction Fetch Unit

Manages the program counter (PC) and instruction memory (IMEM). Selects the next PC based on control signals (sequential, branch, jump, or stall). Outputs the current instruction and PC for the ID stage.

CONTROLLER: Control Unit

Decodes the opcode and generates control signals for the entire pipeline. Recognizes 9 instruction types (R, I, Load, Store, Branch, LUI, AUIPC, JAL, JALR). Handles hazard signals to stall or insert NOPs. Manages branch/jump state machine for delayed branch slots.

REG_BANK: Register File

Stores 32 general-purpose 32-bit registers (x0–x31). Provides dual read ports for rs1 and rs2 in the ID stage. Supports write-back to a destination register in the WB stage. x0 is hardwired to zero.

SIGN_EXTEND: Immediate Extender

Extracts and sign-extends immediate values from instructions. Handles different immediate formats (12-bit for I/S/B types, 20-bit for U/J types). Outputs a 32-bit signed immediate for ALU and address calculations.

ALU: Arithmetic Logic Unit

Performs all computations: ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, and comparisons (EQ, NE, LT, GE, LTU, GEU). Selects operands from register file or immediate values via mux logic. Integrates data forwarding paths to bypass earlier pipeline stages. Outputs the result (for R/I instructions) or computed address (for branches/loads/stores).

HAZARD_DETECTION_UNIT: Hazard Detector

Monitors the ID/EX and IF/ID latches to detect load-use dependencies. Signals when a stall is needed: if a load instruction is in EX and the next instruction tries to use the loaded register. Triggers pipeline freeze and NOP injection.

FORWARDING_UNIT: Data Forwarding Logic

Compares source operands (rs1, rs2) of the current EX stage instruction with destination registers (rd) of the MEM and WB stage instructions. Selects which result path to use: register file, EX result, or MEM result. Respects OP_1 and OP_2 signals (not all operands can be forwarded, e.g., PC is not forwardable).

MEM: Data Memory

Stores program data and variables. Supports read operations (for loads) and write operations (for stores). Initializes from a hex file for testing.

Pipeline Latches (IF_ID, ID_EX, EX_MEM, MEM_WB): Inter-stage Buffers

Stores results and control signals between pipeline stages. Updated on clock edges; can be frozen (FREEZE signal) or flushed (NOP_* signals). Ensures data consistency across clock domains and simplifies stage boundaries.

Key Control Signals

The controller outputs a set of control signals that orchestrate the entire pipeline. Understanding these signals is key to grasping how instructions flow through the design:

TYPE: Instruction type (R, I, Load, Store, Branch, etc.). Used for decoding and routing.

ALU_OP: ALU operation code (ADD, SUB, XOR, AND, OR, SLL, SRL, SRA, comparisons, etc.).

OP_1, OP_2: ALU input selection: 00=register, 01=PC, 10=0 or imm. Allows flexible operand routing.

MEM_READ: Enable data memory read (for load instructions).

MEM_WRITE: Enable data memory write (for store instructions).

WE: Write-enable for the register file. Controls whether the result updates rd.

PC_SRC: Program counter selection: 00=hold, 01=sequential (+4), 10=branch offset, 11=jump target.

FREEZE: Stall signal. Freezes IF/ID and ID/EX latches to prevent new instructions from entering the pipeline.

NOP_IF_ID, NOP_ID_EX: NOP (no-op) injection. Converts instruction to ADD x0, x0, x0 to preserve pipeline state during stalls or branch delays.

UNSIGNED: Unsigned comparison flag. Distinguishes between signed (LT, GE) and unsigned (LTU, GEU) comparisons.

MEM_WRITE_SEL: Selects which data source to write to memory (for distinguishing load vs. store paths).

Supported Instruction Types

R-Type: *add, sub, and, or, xor, sll, srl, sra, slt, sltu*

Register-register operations. Two source operands, one destination.

I-Type: *addi, andi, ori, xori, slli, srli, srai, slti, sltiu, jalr*

Register-immediate operations. One register source, one immediate source.

S-Type: *sw, sh, sb*

Store to memory. Register value written to address calculated from base register + offset.

Load-Type: *lw, lh, lb, lhu, lbu*

Load from memory. Value from address (register + offset) written to destination register.

B-Type: *beq, bne, blt, bge, bltu, bgeu*

Conditional branch. Compares two registers; jumps if condition is true.

U-Type: *lui, auipc*

20-bit immediate operations. LUI loads immediate into upper bits; AUIPC adds PC to immediate.

J-Type: *jal*

Unconditional jump. Stores return address (PC + 4) and jumps to target (PC + offset).

File Structure & Usage

The project is organized into modular Verilog files, each containing a specific functional block. This modular approach makes the code easier to understand, test, and maintain.

ALU.v: Arithmetic Logic Unit with data forwarding mux logic.

CONTROLLER.v: Control unit that decodes opcodes and generates control signals.

EX_MEM.v: Pipeline latch between EX and MEM stages.

FORWARDING_UNIT.v: Detects and enables data forwarding paths.

HAZARD_DETECTION_UNIT.v: Detects load-use hazards and triggers pipeline stalls.

ID_EX.v: Pipeline latch between ID and EX stages.

IF_ID.v: Pipeline latch between IF and ID stages.

IF_MODULE.v: Instruction fetch unit with PC management and instruction memory.

MEM.v: Data memory module with read/write logic.

MEM_WB.v: Pipeline latch between MEM and WB stages.

REG_BANK.v: Register file with dual read and single write port.

SIGN_EXTEND.v: Immediate value extender for different instruction types.

IM.hex, MEM.hex, REG_BANK.hex: Initialization files for instruction, data, and register memories.

rv32i_wave.vcd, sim.vvp: Simulation artifacts (waveform and executable).

Testing & Verification

The processor is verified through simulation with Icarus Verilog. A custom toolchain is used to compile C programs (without libc) into RV32I machine code. The generated hex files are loaded into instruction memory, and the simulation runs until completion. Waveforms can be examined to trace instruction flow through the pipeline. A complete test suite includes:

- Basic arithmetic and logical operations (add, sub, and, or, xor, shifts)
- Register-immediate operations (addi, slti, etc.)
- Memory operations (load, store with different widths)
- Conditional branches (beq, bne, blt, etc.)
- Jumps (jal, jalr) with return address handling

- Complex algorithms (Bellman-Ford, etc.)
- Data forwarding scenarios (consecutive dependent instructions)
- Load-use stall scenarios (immediate dependency on load result)

Design Decisions & Trade-offs

Delayed Branches:

Branch outcomes are determined in the EX stage (2 cycles after fetch). Rather than flush the pipeline, we use a one-cycle delay slot. This simplifies control logic but adds latency. Modern high-performance designs use branch prediction.

No Branch Prediction:

The processor assumes branches are not taken (fall-through). This is simple but can stall the pipeline when branches are frequently taken.

Harvard Architecture:

Separate instruction and data memory avoid structural hazards. This is common in embedded systems and simplifies memory bandwidth analysis. A unified memory would require more complex arbitration logic.

Full Data Forwarding:

Two forwarding paths (from EX and MEM stages) eliminate many stalls. Only load-use dependencies require a stall. This balances complexity with performance.

RV32I Base Set:

Supports only the base integer ISA, no floating-point, atomic, or vector extensions. This keeps the design tractable for educational purposes. Extensions would require additional ALU operations and control logic.

Fixed Latency Memory:

Memory reads and writes complete in one cycle. Real systems have variable latency and caches. This simplification avoids complex cache coherency logic.

Lessons Learned & Debugging

Pipeline Latch Timing:

Getting the timing of pipeline latches right is crucial. Non-blocking assignments ($\leq$) must be used consistently to avoid race conditions between stages.

NOP Injection Encoding:

NOPs are implemented as ADD x0, x0, x0 (a harmless operation). This encoding must match the ISA exactly, or the pipeline will execute unintended instructions.

Hazard Detection Priority:

Load-use hazards require a stall, but data forwarding from other stages can proceed. The hazard unit must carefully distinguish these cases.

State Machine Correctness:

The controller uses a state machine for branch/jump delay slots. Testing all state transitions (especially with concurrent hazards) is essential.

Sign Extension Edge Cases:

Different instruction types have different immediate formats (12-bit, 20-bit). Incorrect extraction or sign extension leads to wrong branch targets or operand values.

Memory Byte Ordering:

RISC-V uses little-endian byte order. When loading bytes (lb, lbu), the offset matters. A detailed testbench that inspects memory state helps catch these bugs.

Performance Characteristics & Future Work

Current Performance:

The baseline design achieves about 1 instruction per cycle (IPC) on code with minimal data dependencies. Load-use hazards reduce this to ~0.8 IPC in typical programs. Branch mispredictions (assumed not taken) add latency. Real-world performance would be measured with actual benchmarks and hardware traces.

Potential Enhancements:

- Branch prediction with a simple 1-bit or 2-bit predictor to reduce branch latency.
- Deeper pipeline (7–9 stages) for higher frequency, with more sophisticated hazard handling.
- Out-of-order execution with a reservation station for higher IPC.
- Cache hierarchy (L1 instruction and data caches) for realistic memory latency.
- Floating-point extension (F) to support real-world scientific code.
- Atomic instructions (A extension) for multi-threaded programs.
- MMU (Memory Management Unit) with TLB for virtual memory support.
- Trap handling (exceptions, interrupts) with CSR (Control and Status Registers).

References & Resources

This design draws on classical computer architecture textbooks and the RISC-V ISA specification. Key references include:

- Computer Architecture: A Quantitative Approach by Hennessy & Patterson
- RISC-V Instruction Set Manual (Volume I: Base ISA)
- Digital Design by M. Morris Mano & Michael D. Ciletti
- MIT OpenCourseWare – Computer Architecture (6.004)
- RISC-V GNU Toolchain Documentation

For detailed implementation notes, refer to individual module comments in the Verilog source code. This document provides a high-level overview; studying the code alongside this explanation is strongly recommended.